

Tutorial 5

Graduate Algorithms

Question 1: Counting Inversions

Given an (1-indexed) array A of n distinct integers, give an algorithm to count the number of inversions in $O(n \log n)$ time.

An inversion is a pair of i, j such that:

- ▶ $1 \leq i < j \leq n$
- ▶ $A[i] > A[j]$

Hint: Consider a function

```
def inversions(A: list[int]) ->
    tuple[int, list[int]]
```

where we are returning both the number of inversions and sorted list A .

Question 2: Strassen's Algorithm

Given two $n \times n$ matrices A and B , compute their product AB in time $O(n^{\log_2 7})$.

Part 1: Assuming $m \geq n$, find an $O(m^2 n^{0.81})$ algorithm to multiply A ($m \times n$) and B ($n \times m$) using the above as a subroutine. *Hint: Multiply $\lceil m/n \rceil^2$ pairs of order- n matrices*

Part 2: Assuming $m \geq n \geq t$, find an $O(mnt^{0.81})$ algorithm to multiply A ($m \times n$) and B ($n \times t$) using the above as a subroutine. *Hint: Multiply pairs of $t \times t$ matrices*

Question 3: Merging Arrays

Let $A_1, A_2, \dots, A_k$ be k arrays, each of which has been sorted.

These arrays are mutually disjoint, namely, no integer can appear in more than one array.

Design an algorithm to merge the k arrays into one sorted array in $O(n \log k)$ time, where n is the total length of the k arrays.

Note: These arrays may have different lengths.

Question 4: Max Weight Subarray

Let A be an array of n integers (A is not necessarily sorted). Each integer in A may be positive or negative.

Given i, j satisfying $1 \leq i \leq j \leq n$, define sub-array $A[i : j]$ as the sequence $(A[i], A[i + 1], \dots, A[j])$, and the weight as $A[i] + A[i + 1] + \dots + A[j]$.

Example: $A = (13, -3, -25, 20, -3, -16, -23, 18)$; $A[1 : 4]$ has weight 5; $A[2 : 4]$ has weight -8 .

- (a) Find a sub-array with largest weight among all $A[i : j]$ with $j = n$ in $O(n)$ time.
- (b) Find a sub-array with largest weight in $O(n \log n)$ time (among all possible sub-arrays).

Question 5: L Tiles

Given a grid of $2^N \times 2^N$, all cells are initially empty except one cell. Let the blocked cell be B .

Write an algorithm to figure out a way to fill the grid with L shapes (without overlapping other tiles or B) or report “NO” if that is not possible. Your algorithm should run in $O(4^N)$.

The $N = 2, B = (3, 2)$ case is shown in the problem document.

Hint: We never have to return “NO”.

Question 6: Big Mod

Given p , e , b , compute $p^e \bmod b$ in $O(\log e)$ time.

Question 7: DeadShot (DS Algo Mock-Midsem 2026)

To celebrate the end of midsems, a group of students head to the CLAB to play Deadshot.io. There are n players, and the i -th player has a skill score of $A[i]$.

They want to split into two teams. Rules:

- ▶ Team Kabir must have a total skill score $\geq$ Team Piyush
- ▶ The difference between team scores should be as small as possible

Write a function `balanced_teams(A)` that returns the minimum possible difference between the total skill scores in $O(n2^{n/2})$ time.

Hint: The intended solution uses $O(2^{n/2})$ space as we need *two* arrays of that length.

Question 7 (continued): DeadShot Examples

Input	Output	Explanation
$A = [5, 4, 7, 6]$	0	Team Kabir: $5 + 6 = 11$ Team Piyush: $4 + 7 = 11$
$A = [10, 1, 1]$	8	Team Kabir: 10 Team Piyush: $1 + 1 = 2$
$A = [3, 4, 2, 2]$	1	Team Kabir: $4 + 2 = 6$ Team Piyush: $3 + 2 = 5$

Question 7 (continued): DeadShot Tasks

- (a) Write pseudocode implementing `balanced_teams(A)`. You are not allowed to import libraries or use any language-specific features.
- (b) Argue that your algorithm runs in $O(n2^{n/2})$ time.
- (c) Argue the correctness of your implementation.

